Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition

Anonymous Author(s)

Abstract

Large language models can generate short register-transfer level (RTL) fragments, but benchmark-level hardware synthesis requires executable correctness: simulation success, protocol consistency, and golden-output agreement. ArchXBench exposes this gap through a sharp degradation on hard Level-4-and-above rows under direct pass@5 prompting. This paper evaluates whether verifier-grounded autonomous synthesis can cross that frontier without confusing model failures with benchmark-contract failures. Four conditions are compared on ArchXBench Levels 3–6: direct pass@5 generation (C1), monolithic golden-feedback repair (C2g), decomposition-guided iterative repair (C4i), and testbench-localized decomposition (C4tl). Under GPT-5.5, C4i solves five Level-3 numerical kernels on all three trials, while direct pass@5 and C2g do not achieve a full-row solve. C4tl solves all four core Level-4 designs, including 16-point FFT/IFFT, on five independent trials. Golden-verified Level-5 and Level-6 solves are obtained on convolution, image-processing, and AES designs. Decomposition is not universally superior: C2g solves several Level-5 rows that decomposed methods do not. The central contribution is an executable-contract-aware synthesis methodology: original-benchmark solves, minimally repaired executable contracts, and held rows are separated before success is claimed. Code, prompts, logs, generated RTL, and result JSONs are prepared for anonymous artifact release.[1]

CCS Concepts

• **Hardware** → **Hardware description languages and compilation**; **Electronic design automation**; • **Software and its engineering** → *Automatic programming*.

Keywords

benchmark evaluation, CEGIS, hardware verification, LLM agents, RTL synthesis, Verilog generation

ACM Reference Format:

Anonymous Author(s). 2027. Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition. In *Proceedings of 32nd Asia and South Pacific Design Automation Conference (ASP-DAC '27).* ACM, New York, NY, USA, 6 pages.

1 Introduction

Autonomous code agents close executable loops: generate a candidate, run a checker, inspect failures, and repair. For software tasks, this loop has changed what counts as a meaningful benchmark result. For hardware design, the question is harder: can an autonomous agent synthesize RTL that survives simulation, protocol assumptions, and golden-output comparison?

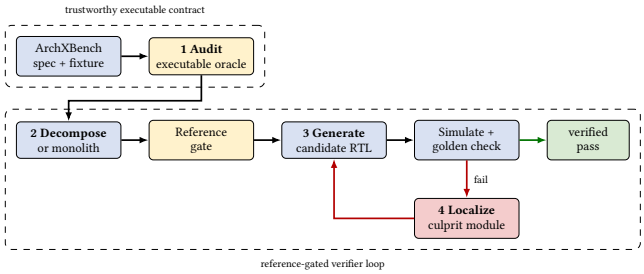

Figure 1: C4tl pipeline: audit the executable contract, validate references, then repair only the localized culprit module.

RTL correctness depends on bit widths, signedness, clocking, reset protocols, pipeline latency, and streaming handshakes; a design may compile yet remain unusable because it mishandles these semantics. Hard RTL synthesis therefore tests low-level, state-dependent, timing-sensitive generation rather than surface-level code completion.

This paper studies ArchXBench, which spans arithmetic, FFTs, filters, image processing, matrix multiplication, and cryptography across six levels [8]. Unlike the smaller VerilogEval and RTLLM tasks [4, 5], its hard Level-4 FFT/IFFT rows remain unsolved under reported zero-shot pass@5. Solving them changes the question from direct sampling to whether a verifier-grounded agent can converge to correct RTL.

We ask whether verifier-grounded synthesis can solve hard RTL tasks while separating model failures from benchmark-contract failures. Under GPT-5.5, C4i solves five Level-3 kernels on all three trials, and C4tl solves all four core Level-4 designs on five trials, including FFT/IFFT. Golden-verified solves extend through Level 6, while strong C2g results show that decomposition does not dominate monolithic repair. Together, the results cross parts of the reported frontier and show why executable contracts must be audited before interpreting success or failure.

This paper makes three contributions:

(1) A reference-gated modular workflow validates model-generated scaffolds and localizes failing compositions. C4tl robustly solves original-contract 16-point FFT/IFFT rows that fixed-order C4i does not, with C2g as a strong matched baseline.
(2) An executable-contract-aware discipline separates released, minimally repaired, and held contracts so benchmark defects are not scored as model progress.
(3) A four-condition comparison shows decomposition gains on Level-3 kernels and over fixed-order repair on Level-4 FFT/IFFT, while monolithic repair remains competitive or stronger on compact Level-5/Level-6 tasks.

[1] Repository URL withheld for double-blind review.

2 Related Work

2.1 LLM-Based RTL Generation

VerilogEval introduced an HDLBits-style Verilog benchmark with automatic functional checking, while RTLLM created an open benchmark for design RTL generation with larger natural-language tasks [4, 5]. Gai et al. study fine-tuned code models and iterative feedback for HLS-oriented C/C++ generation [1]; our focus is hard RTL under released executable contracts. Subsequent systems move beyond direct prompting in several directions. AutoChip uses compiler and simulator feedback for iterative HDL generation [10]. VerilogCoder adds graph-based planning and AST-based waveform tracing [2]. AutoVeriFix corrects simulation discrepancies in LLM-generated Verilog [9], and Spec2RTL-Agent studies agentic specification-to-RTL generation [14]. VFlow searches over agentic workflows using MCTS [13], HDLxGraph retrieves from HDL graph databases [15], and REvolution/COEVO use evolutionary search for RTL correctness and optimization [6, 7]. Recent benchmark and verification work further shows that flawed executable contracts and weak oracles can distort RTL evaluation [3, 11, 12]. Table 1 summarizes where the present study sits.

Our work evaluates on ArchXBench Levels 3–6, including rows at and above the reported Level-4 frontier. It separates original benchmark contracts from repaired executable contracts so that model failures are not confused with stale testbenches, file-format mismatches, or inconsistent golden comparators.

2.2 Verifier-Grounded Synthesis

Counterexample-guided inductive synthesis (CEGIS) uses a verifier to reject incorrect candidates and guide the next attempt. LLM-based repair loops instantiate a related pattern: generate code, run tests, inspect errors, and repair. For RTL, verifier feedback is heterogeneous: failures arise from syntax, elaboration, simulation, timing, protocol mismatch, file-output mismatch, or incorrect golden references. The present pipeline treats simulator and golden-output feedback as first-class signals and audits the benchmark contract when executable artifacts disagree.

3 Problem Setting

Each ArchXBench design provides a natural-language specification and executable assets: Verilog testbenches, input files, output files, and comparison scripts. An autonomous synthesis condition receives the task and emits RTL. For self-checking designs, a run succeeds only when the official testbench passes all checks. For file-output designs, simulation completion is not sufficient; the generated output must match the golden output element by element.

Three evaluation classes are distinguished:

- **Original-contract results:** runs against the released benchmark as-is.
- **Repaired-contract results:** runs against a copied fixture with minimally repaired infrastructure inconsistencies.
- **Held or excluded rows:** tasks where the released contract remains too ambiguous for a credible positive result.

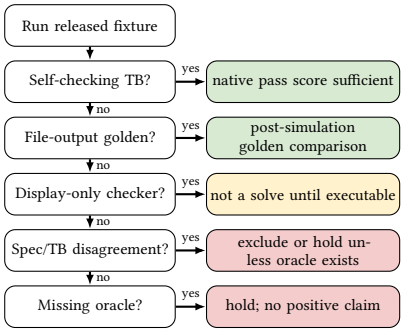

Figure 2: Executable-contract classification. Positive results are claimed only when the released or repaired fixture has an executable oracle.

4 Method

4.1 Evaluated Conditions

All conditions receive the same specification and interface and use the same oracle; to match direct pass@5, only C1 does not see the testbench. The comparison stages whole-design feedback (C2g), fixed-order decomposition (C4i), and stricter reference gating, reference-guided regeneration, and localized repair (C4tl). Because C4tl bundles these mechanisms, it is evaluated as a complete condition rather than a factorized localization ablation.

Condition grid.

Cond.	Structure	Feedback and repair
C1	monolith	direct pass@5; no feedback
C2g	monolith	golden feedback; whole-design repair for 30 rounds
C4i	decomp.	sanity reference gate; fixed-order module repair, ≈28 rounds
C4tl	decomp.	strict reference gate; localize culprit, then repair only that module, 3+≈28 rounds

C1: direct generation. The model emits five independent RTL candidates from the specification and interface; the highest-scoring is retained without repair.

C2g: monolithic golden-feedback repair. The model repairs a complete top module for up to 30 rounds using compile errors, self-check failures, and indexed golden mismatches. This strong baseline uses the same oracle as decomposed methods without module-integration risk.

C4i: decomposition-guided iterative repair. C4i decomposes the task into 3–7 interfaced submodules, a top wrapper, and references, then solves modules in fixed order after a composition sanity check. Where possible, submodules are combinational, with state and protocol control in the wrapper; pipeline positions constrain local implementations. During repair, neighbors are held at their references, the approximately 28-call budget is divided across modules, and indexed golden mismatches provide executable feedback.

C4tl: testbench-localized decomposed repair. C4tl uses the same decomposition structure as C4i but adds two mechanisms: a strict reference gate and trace-lifted fault localization.

Table 1: Positioning relative to recent LLM-aided RTL work. The key distinction is a hard-ArchXBench study that combines verifier-grounded synthesis with explicit original-contract versus repaired-contract accounting.

Line of work	Examples	Main strength	Difference from this study
RTL benchmarks	VerilogEval, RTLLM	Automatic functional checks	Smaller or lower-frontier tasks; not focused on ArchXBench hard-level contracts
Agentic RTL generation	AutoChip, VerilogCoder, AutoVeriFix, Spec2RTL	Strong agents with feedback, planning, tracing	Does not separate original, repaired, and held ArchXBench contracts
Workflow/search	VFlow, REvolution, COEVO	Workflow, population, and optimization search	Search-oriented; does not study benchmark-contract validity
Benchmark/oracle validity	RTL-BenchMT, FormalRTL, RTL-BenchLS	Maintenance, stronger oracles, formal checks	Validates contract concerns; not focused on ArchXBench hard rows
This work	C1/C2g/C4i/C4tl on ArchXBench	Golden repair, decomposition, localization	Separates original, repaired, and held rows while crossing hard-frontier tasks

Table 2: Executable-contract-aware verifier-grounded RTL synthesis. The condition parameter selects C1, C2g, C4i, or C4tl by enabling feedback, decomposition, reference gating, and localization.

Input	Specification S, released fixture B, condition c, trial ID r, budget T
Output	SOLVED, UNSOLVED, or HELD, with saved artifact record
Audit	Classify B using Fig. 2. If no executable oracle can be recovered, return HELD.
Decompose	If c uses decomposition, request interfaces, top wrapper, and reference modules $R_1, \ldots, R_n$; otherwise set $n = 1$ and target the whole design.
Gate	When enabled, simulate the composed references against B; retry with failure feedback and abort if no valid reference composition is found.
Generate	Produce candidate RTL modules $M_1, \ldots, M_n$ from the context available to condition c and trial r.
Verify	For each round $t \le T$, simulate the composed candidate and compute pass/golden score. If all checks pass, save RTL and return SOLVED.
Localize	Select a repair target: C2g selects the whole design; C4i selects the next module in fixed order; C4tl scores each M_i with all non-target modules held at their references and selects the lowest-scoring module.
Repair	Repair only the selected target using compile errors, failing indices, expected/actual values, isolation scores, and the relevant reference; then repeat verification.
Exit	Save the final artifact and return UNSOLVED.

References are generated online by the same model from public task inputs; no benchmark solution RTL is supplied, and generation or regeneration counts toward the budget. They serve as internal scaffolds, so C4tl is evaluated as a bundled reference-guided condition rather than an isolated localization ablation.

The reference gate requires the composed reference implementation to pass the system-level testbench before synthesis begins; failed references are retried up to three times, otherwise the run aborts. After candidate generation, C4tl localizes a failure by evaluating each candidate submodule in a reference-isolated composition: the target remains a candidate while every non-target submodule is held at its validated reference. Let $M^{[i]} = \mathrm{Compose}(R_1, \ldots, R_{i-1}, M_i, R_{i+1}, \ldots, R_n)$ denote this composition. For executable score function $q(\cdot)$, C4tl computes

$$q_i = q(M^{[i]}), \qquad i^\star = \arg\min_i q_i.$$

The selected module $M_{i^\star}$ is repaired using simulator output, golden mismatches, isolation scores, and a prompt to trace the first failing datapath against its reference.

Unlike C4i's fixed order, C4tl uses reference-isolated scores to target the most implicated module and avoid diffuse repair. This heuristic is strongest for a valid reference with one dominant fault, but coupled bugs, poor decompositions, or masking can mislead it.

4.2 Artifact and Verification Discipline

Each artifact records the design, condition, model, trial identifier (the legacy seed field), scores, LLM calls, wall time, and RTL. File outputs require exact cardinality and the validated comparator; simulation completion alone is not a solve.

5 Experimental Setup

5.1 Benchmark Scope

The evaluation covers ArchXBench Levels 3–6, from iterative arithmetic through FFTs, image processing, matrix multiplication, and AES, including the reported transition to hard Level-4-and-above designs. All 28 released Level-3–6 rows were audited before outcome aggregation: 16 retained credible released contracts, 11 required separately labelled and independently validated copied fixtures, and one remains held because no unambiguous executable contract could be recovered. Repository scripts compile each candidate with Icarus Verilog before self-checking or golden comparison.

5.2 Trials, Model, and Metrics

Main tables use stochastic trials 42, 123, and 456; because the provider exposes no sampling control, these are replication identifiers, not model seeds. Trials 789 and 1024 test C4tl Level-4 robustness. Each C1 trial is pass@5, with five such trials rather than three for `fp_adder` and `fp_multiplier`; the primary model is GPT-5.5.

For run r, let p_r/t_r denote native self-checking passes and g_r/h_r denote golden-output matches. The executable score is

$$q(r) = \begin{cases} g_r/h_r, & h_r > 0, \\ p_r/t_r, & h_r = 0 \text{ and } t_r > 0, \\ 0, & \text{otherwise.} \end{cases}$$

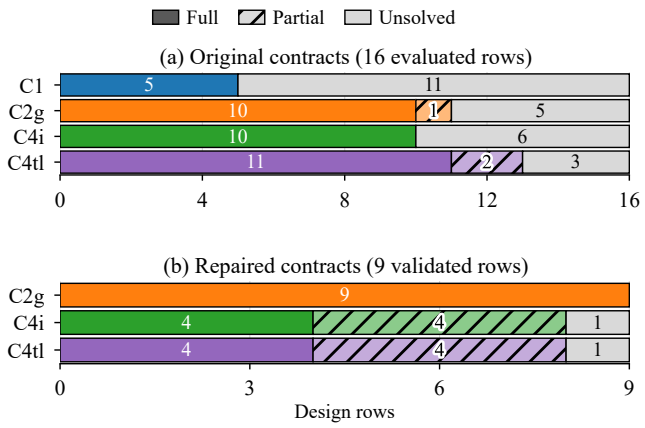

Figure 3: Aggregate verified outcomes by contract class. Full rows solve all main trials; partial rows solve at least one but not all. C1 was not evaluated on repaired fixtures.

Run r solves iff $q(r) = 1$. For design d, condition c, and trial set R, full-row and partial-row indicators are

$$F_{d,c} = \prod_{r \in R} \mathbf{1}[q(d,c,r) = 1], \qquad P_{d,c} = \mathbf{1}\left[\max_{r \in R} q(d,c,r) = 1\right] - F_{d,c}.$$

Figure 3 summarizes the corresponding verified design-row outcomes separately for original and repaired contracts. Unlike an any-sample pass@k count, a full row requires every independent trial to pass the exact executable oracle; partial rows expose rather than absorb stochastic success.

Under C4tl, the Level-4 FFT solves in roughly 5 minutes and 6 LLM calls on average. Failed decomposed FFT/IFFT diagnostics consume about 26–29 calls and 20–45 minutes.

6 Crossing the Original-Contract Frontier

Across the 16 original-contract rows, the full/partial counts are 5/0 for C1, 10/1 for C2g, 10/0 for C4i, and 11/2 for C4tl; Table 3 gives the per-design results.

Level 3. Because prior models solve some Level-3 tasks under direct pass@5, this is a matched-model robustness comparison, not an unreachability claim. Under GPT-5.5, C4i solves five kernels on all trials, versus C1's 0/3–0/5 and C2g's 0/3–1/5. C4tl matches C4i on four of five rows but solves `gauss_siedel` at only 1/3. The negative result on `newton_raphson_polynomial` is an original-checker ceiling: three of its 100 assertions are structurally unsatisfiable, capping the achievable score at 97/100. After removing only those three unsatisfiable residual assertions in the repaired-contract fixture, C2g solves all three trials (Table 5), confirming that the design itself is within model capability.

Level 4. C4tl solves all four core Level-4 rows on 3/3 main trials and 2/2 robustness trials, for 5/5 total. The self-checking FFT/IFFT rows require no benchmark repair and are the strongest frontier evidence. In one trial, C4tl decomposes FFT into bit-reversal, twiddle, butterfly, and scaling modules, then localizes a failed composition before repair. C2g also solves FFT/IFFT at 3/3, while C4i does not; C2g avoids module order monolithically, whereas C4tl retains modular isolation. The pipeline rows (`fp_adder_pipeline`, `fp_mult_pipeline`) are solved by all conditions.

Levels 5–6. C4i and C4tl obtain 3/3 golden-verified solves on `conv1d` and both AES rows; AES uses six isolated submodules from S-box through key expansion. At Level 5, C2g alone fully solves `conv2d` and DCT/IDCT, with 4,096/4,096 and 16/16 matches. It also solves `unsharp_mask` (65,536/65,536), while C4tl solves one trial. Monolithic repair is strongest when whole-design feedback is tractable or the difficulty is a global format or encoding convention. `harris_corner_detection` is reported only under its repaired contract in Table 5 because the released checker does not provide a credible exact oracle.

The results are not presented as a universal decomposition win. C2g is a serious baseline: it matches the strongest decomposed result on all four core Level-4 rows and both AES rows, and matches or exceeds it on all four Level-5 rows. The most accurate interpretation is that decomposition provides value on specific hard rows and gives a structured synthesis process, while monolithic golden-feedback repair is often sufficient and sometimes better.

Model generality. To test whether the hard-frontier result is specific to `gpt-5.5`, Table 4 repeats the strongest original-contract rows with Claude Sonnet 5. The original Level-4 FFT/IFFT rows solve on all three trials under both C2g and C4tl, and the Level-6 AES rows solve on all three trials under C2g with golden verification. Sonnet 5 does not solve AES under C4tl because the reference decomposition fails before localized repair can begin. For AES encryption, Sonnet 5 produces decompositions whose reference compositions fail the original system testbench; for AES decryption, it does not produce a usable reference composition within the generation budget. This indicates model-dependent sensitivity in the decomposition gate, while monolithic golden-feedback repair remains robust on the same AES rows.

Case study: original Level-4 FFT. `fft_16pt_iterative` is a self-checking original-contract row, so no benchmark repair is involved. C1 fails on all three main trials, while C2g and C4tl solve all three. The row therefore isolates the central mechanism: executable feedback can turn a direct-generation failure into a verified RTL solve without changing the released checker. In one robustness trial, C4tl localizes a 9/33 failure to the butterfly while the twiddle ROM, index generator, and bit-reversal modules each pass in isolation; one repair then passes all 33 checks. The repaired RTL explicitly implements signed Q1.15 complex rotation.

7 Executable-Contract Audit

Hard RTL benchmarks are useful only when their executable contracts reflect the intended task. During evaluation, several rows were found to contain inconsistent specifications, stale comparators, display-only checkers, or output-schema mismatches. The released benchmark is not overwritten. Repaired fixtures are kept under a separate artifact root and reported separately.

The audit follows a fixed protocol. First, the released fixture is run unchanged. Second, the checker is classified (self-checking, file-output with golden, display-only, stale, specification-inconsistent, or missing-oracle). Third, file-output rows are judged only by post-simulation golden comparison. Fourth, infrastructure repairs are

Table 3: Original-contract results. Cells show full, partial, and failed rows; GV marks golden verification. Coverage is 6/6, 4/7, 4/6, and 2/9 released L3–L6 rows. Each C1 trial is pass@5; L4 C4tl additionally solves trials 789 and 1024 (5/5 total).

Level	Design	C1	C2g	C4i	C4tl	Evidence
L3	fp_adder	0/5	1/5	**3/3**	**3/3**	decomposition wins
L3	fp_multiplier	0/5	0/3	**3/3**	**3/3**	decomposition wins
L3	gauss_siedel	0/3	0/3	**3/3**	1/3	C4i strongest
L3	gradient_descent	0/3	0/3	**3/3**	**3/3**	decomposition wins
L3	newton_raphson_sqrt	0/3	0/3	**3/3**	**3/3**	decomposition wins
L3	newton_raphson_polynomial	0/3	0/3	0/3	0/3	checker ceiling at 97/100
L4	fft_16pt_iterative	0/3	**3/3**	0/3	**3/3**	hard-row solve; 5/5
L4	ifft_16pt_iterative	0/3	**3/3**	0/3	**3/3**	hard-row solve; 5/5
L4	fp_adder_pipeline	**3/3**	**3/3**	**3/3**	**3/3**	all methods solve; 5/5
L4	fp_mult_pipeline	**3/3**	**3/3**	**3/3**	**3/3**	all methods solve; 5/5
L5	conv1d	**3/3**	**3/3**	**3/3**	**3/3**	GV; all methods solve
L5	conv2d	0/3	**3/3**	0/3	0/3	GV; C2g strongest
L5	dct_idct_8pt_pipelined	0/3	**3/3**	0/3	0/3	GV; C2g strongest
L5	unsharp_mask	0/3	**3/3**	0/3	1/3	GV; C2g strongest
L6	aes_encryption	**3/3**	**3/3**	**3/3**	**3/3**	GV; all methods solve
L6	aes_decryption	**3/3**	**3/3**	**3/3**	**3/3**	GV; all methods solve

Table 4: Second-model validation with Claude Sonnet 5 on selected hard-frontier rows. GV denotes golden-verified file-output rows.

Design	C2g	C4tl
fft_16pt_iterative	**3/3**	**3/3**
ifft_16pt_iterative	**3/3**	**3/3**
aes_encryption	**3/3** GV	0/3 GV
aes_decryption	**3/3** GV	0/3 GV

applied only in copied fixtures and only when the intended oracle is recoverable from released assets. Fifth, unresolved rows are held rather than converted into positive results. The repair rule is intentionally narrow: a repaired fixture may make an existing oracle executable or repair file-format infrastructure, but it may not change the semantic task, introduce hidden target behavior, or move a row into the original-contract table. Every repaired oracle is validated against an independent golden reference before any synthesis method runs; accepted fixtures must reach full oracle score (for example, 1000/1000 FIR samples or 97/97 repaired Newton assertions). Fixtures that cannot be validated this way are discarded or held.

The released harris_corner_detection testbench instantiates a 256×256 image although the public interface and released files contain $128 \times 128 = 16{,}384$ samples; its ±1 comparator also accepts every binary 0/1 mismatch. The repaired fixture restores the released dimensions, exact output cardinality, and exact binary comparison, so Harris is reported only below. The repaired fp_low_pass_fir row uses an oracle recovered from official upstream history and independently validated; the released fixture remains held.

The audit produces both positive and negative evidence. C2g solves all nine repaired rows in Table 5 at 3/3; C4i and C4tl also fully solve conv_3d, multich_conv2d, quantized_matmul, and systolic_gemm, and obtain partial FP-FIR and Newton coverage. Harris remains monolithic-only under its exact repaired checker. On corrected Q1.15 Level-4 FIR fixtures, C2g solves band-, high-, and low-pass rows at 3/3, 2/3, and 2/3, respectively, while C4tl remains

Table 5: Repaired-contract results. These are separate from original ArchXBench claims.

Design	C2g	C4i	C4tl
conv_3d	3/3	3/3	3/3
multich_conv2d	3/3	3/3	3/3
quantized_matmul	3/3	3/3	3/3
harris_corner_detection	3/3	0/3	0/3
fp_band_pass_fir	3/3	1/3	2/3
fp_high_pass_fir	3/3	1/3	1/3
fp_low_pass_fir	3/3	1/3	2/3
newton_raphson_polynomial	3/3	2/3	2/3
systolic_gemm	3/3	3/3	3/3

0/3 on all three; no decomposed condition adds a full-row solve. These repaired results remain separate from the original-contract frontier.

Several rows remain held or excluded. The original L4 FIR family is excluded because the specification and testbench disagree on filter coefficients; only the corrected copied fixtures above are reported. The L6 fft_streaming_64pt is excluded because the released input/output contract has unresolved schema and numeric-encoding ambiguities.

8 Discussion

8.1 When Decomposition Helps and When It Does Not

Decomposition is most effective when a design factors naturally into modules and when failure localization reduces the repair burden. The Level-3 numerical kernels illustrate the strongest decomposition-over-monolith wins, while the Level-4 FFT/IFFT rows show a different effect: both C2g and C4tl cross the direct-generation frontier, but fixed-order C4i fails. One explanation is that C4i's fixed per-module repair order is less effective for tightly coupled pipeline stages, whereas the end-to-end C4tl workflow combines validated reference scaffolds, reference-guided module regeneration, and targeted repair. A randomized-order C4i comparison improves FFT/IFFT from 0/6 to 3/6 solved trials, but remains below C4tl's 6/6 main-trial success. This indicates

that repair order contributes to the gap, but does not isolate the remaining advantage among C4tl's bundled components.

By contrast, C2g fully solves several Level-5 rows that neither C4i nor C4tl solves at 3/3. This happens when the design is compact enough for whole-design repair, when module boundaries introduce integration risk, or when the primary difficulty is a global convention such as file format or fixed-point encoding. The paper's claim is therefore conservative: decomposition is a powerful synthesis strategy for some hard rows, but monolithic repair remains strong and should be treated as a primary baseline.

8.2 Why Benchmark Contracts Matter

LLM-aided RTL synthesis papers increasingly rely on executable benchmarks. If the executable contract is ambiguous, a model can fail for the wrong reason or pass the wrong task; separating original-contract and repaired-contract results is therefore a prerequisite for credible evaluation.

9 Limitations

The results do not establish universal dominance over monolithic repair and are limited to fixed-prompt RTL synthesis. Repaired-contract evaluation necessarily involves judgment about infrastructure versus semantic changes; we mitigate this by validating repaired oracles independently, reporting them separately, and holding unresolved rows. C4tl bundles model-generated references, reference-guided regeneration, and localized repair, so the comparison evaluates the complete workflow rather than isolating each component.

10 Conclusion

This study shows that the hard RTL frontier is not fixed by direct generation alone. On ArchXBench, verifier-grounded repair crosses original-contract Level-4 FFT/IFFT rows and obtains golden-verified Level-5/Level-6 solves, while repaired-contract experiments expose additional benchmark tasks that were previously unmeasurable. The results also show why the comparison must be disciplined: monolithic golden-feedback repair is a strong baseline, decomposition helps selectively, and ambiguous executable contracts can turn both failures and successes into misleading evidence.

Hard autonomous RTL synthesis requires structured generation, executable feedback, and trustworthy benchmark contracts together; without all three, both successes and failures can be misinterpreted.

References

[1] Jiahao Gai, Hao (Mark) Chen, Zhican Wang, Hongyu Zhou, Wanru Zhao, Nicholas Lane, and Hongxiang Fan. 2025. Exploring Code Language Models for Automated HLS-based Hardware Generation: Benchmark, Infrastructure and Analysis. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 988–995. doi:10.1145/3658617.3697616

[2] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[3] Kezhi Li, Min Li, Xiangyu Wen, Shibo Zhao, Jieying Wu, Junhua Huang, and Qiang Xu. 2026. FormalRTL: Verified RTL Synthesis at Scale. *arXiv preprint arXiv:2603.08738* (2026).

[4] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design.*

[5] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. *arXiv preprint arXiv:2308.05345* (2023).

[6] Kyungjun Min, Kyumin Cho, Junhwan Jang, and Seokhyeong Kang. 2026. REvolution: An Evolutionary Framework for RTL Generation Driven by Large Language Models. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[7] Heng Ping, Peiyu Zhang, Shixuan Li, Wei Yang, Anzhe Cheng, Shukai Duan, Xiaole Zhang, and Paul Bogdan. 2026. COEVO: Co-Evolutionary Framework for Joint Functional Correctness and PPA Optimization in LLM-Based RTL Generation. *arXiv preprint arXiv:2604.15001* (2026).

[8] Suresh Purini, Siddhant Garg, Mudit Gaur, Sankalp Bhat, Sohan Mupparapu, and Arun Ravindran. 2025. ArchXBench: A Complex Digital Systems Benchmark Suite for LLM Driven RTL Synthesis. In *Proceedings of the 7th ACM/IEEE Symposium on Machine Learning for CAD*. 1–10. doi:10.1109/MLCAD65511.2025.11189156

[9] Yan Tan, Xiangchen Meng, Zijun Jiang, and Yangdi Lyu. 2026. AutoVeriFix: Automatically Correcting Errors and Enhancing Functional Correctness in LLM-Generated Verilog Code. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[10] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. In *Proceedings of the ACM/IEEE Design Automation Conference.*

[11] Jing Wang, Shang Liu, Wenji Fang, Yuchao Wu, Yugao Zhu, and Zhiyao Xie. 2026. RTL-BenchLS: A Large-Scale Benchmark for RTL Reasoning and Generation with Large Language Models. *arXiv preprint arXiv:2606.08976* (2026).

[12] Jing Wang, Shang Liu, Hangan Zhou, and Zhiyao Xie. 2026. RTL-BenchMT: Dynamic Maintenance of RTL Generation Benchmark Through Agent-Assisted Analysis and Revision. In *Proceedings of the ACM/IEEE Design Automation Conference.* doi:10.1145/3770743.3804053

[13] Yangbo Wei, Zhen Huang, Huang Li, Wei W. Xing, Ting-Jung Lin, and Lei He. 2026. VFlow: Discovering Optimal Agentic Workflows for Verilog Generation. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[14] Zhongzhi Yu, Mingjie Liu, Michael Zimmer, Yingyan Celine Lin, Yong Liu, and Haoxing Ren. 2025. Spec2RTL-Agent: Automated Hardware Code Generation from Complex Specifications Using LLM Agent Systems. *arXiv preprint arXiv:2506.13905* (2025).

[15] Pingqing Zheng, Jiayin Qin, Fuqi Zhang, Shang Wu, Yu Cao, Caiwen Ding, and Yang Zhao. 2025. HDLxGraph: Bridging Large Language Models and HDL Repositories via HDL Graph Databases. *arXiv preprint arXiv:2505.15701* (2025).